ITDLUPanel v2 - Phân Tích Kỹ Thuật Chi Tiết

Đại học Đà Lạt | © 2026

Slide 1: Tổng Quan Kiến Trúc

ITDLUPanel v2 - Technical Deep Dive

Phân Tích Kỹ Thuật Toàn Diện



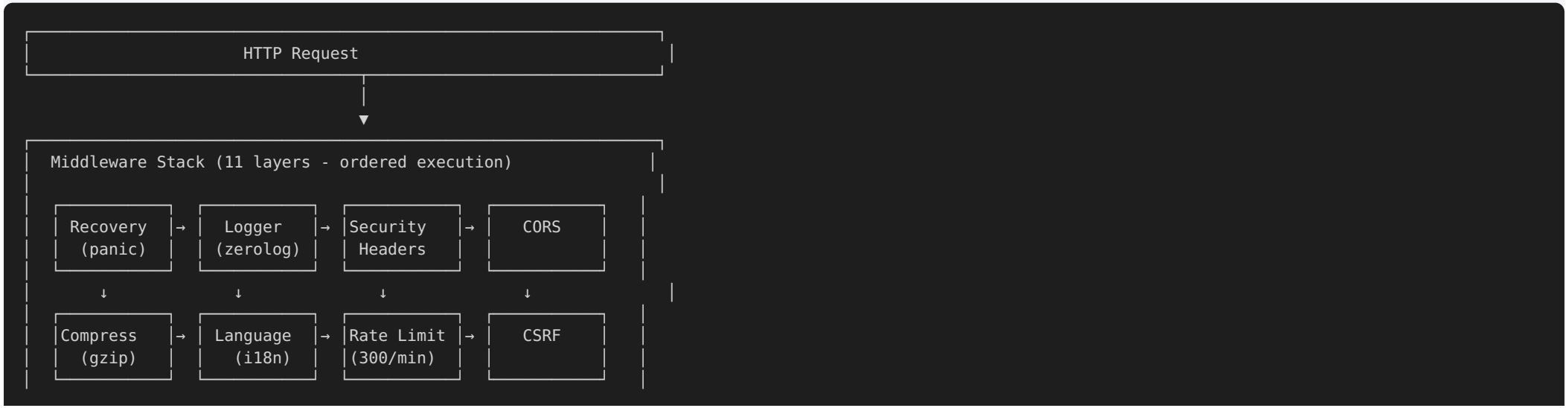
Key Metrics

Metric	Value
Go Files	112 files
Lines of Code	43,314 LOC
API Endpoints	~160 REST + 2 WS
Database Tables	25
Services	33+
Middleware Layers	11

Slide 2: Backend Layer Architecture

Backend Go Architecture

Layered Design Pattern




Audit Logging
Auth (JWT)
RBAC Permission Check
API Handlers (24 files - REST endpoints)
auth
site
database
docker
file
security
cron
backup
... more handlers ...
Service Layer (33+ services)
AuthService
JWT + TOTP
SiteService
CRUD + Stats
DatabaseService
CRUD + Dump
DockerService
46+ methods
FileService
Upload/Download
BackupService
Multi-backend
Data Access Layer
SQLite 1.47
modernc.org
File System
os/exec
Docker Socket
API v1.41

Slide 3: Middleware Stack Chi Tiết

Middleware Pipeline Deep Dive

11-Layer Middleware Stack

Layer 1: Recovery Middleware

```
func RecoveryMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        defer func() {
            if err := recover(); err != nil {
                // Log panic with stack trace
                zerolog.Error().
                    Interface("panic", err).
                    Str("stack", string(debug.Stack())).
                    Msg("Panic recovered")

                c.AbortWithStatusJSON(500, gin.H{
                    "error": "Internal server error",
                })
            }
        }()
        c.Next()
    }
}
```

Layer 2: Structured Logging

```
// zerolog with lumberjack rotation
zerolog.TimeFieldFormat = zerolog.TimeFormatUnix
log := zerolog.New(os.Stdout).
    Level(zerolog.InfoLevel).
    With().
    Timestamp().
    Caller().
    Logger()
```

Layer 3-6: Security Layers

Layer	Header	Purpose
Security	X-Frame-Options	Clickjacking protection
Security	X-Content-Type-Options	MIME sniffing prevention
Security	X-XSS-Protection	XSS filter (legacy)
Security	Strict-Transport-Security	Force HTTPS
CORS	Access-Control-*	Cross-origin requests

Layer 7: Rate Limiting

```
Algorithm: Token Bucket
├── Rate: 300 requests/minute
├── Per: IP address
└── Response: 429 Too Many Requests
```

Layer 8: CSRF Protection

```
┌──────────────────────────────────────────────────────┐
│  CSRF Token Flow                                     │
│                                                      │
│  1. Server generates: csrf_token = crypto            │
│  2. Set-Cookie: HttpOnly; SameSite=Strict            │
│  3. Response includes: X-CSRF-Token header           │
│  4. Client must send both cookie + header            │
│  5. Server validates: cookie == header               │
└──────────────────────────────────────────────────────┘
```

Layer 9: Audit Logging

```
// Write operation audit trail
type AuditLog struct {
    UserID    uint      `db:"user_id"`
    Username  string    `db:"username"`
    Action    string    `db:"action"`     // CREATE, UPDATE, DELETE
    Resource  string    `db:"resource"`   // sites, databases, users
    Details   string    `db:"details"`    // JSON payload
    IPAddress string    `db:"ip_address"`
    Timestamp time.Time `db:"created_at"`
```

```
}
```

Layer 10-11: Auth + RBAC

```
// JWT Token Structure
type Claims struct {
    UserID      uint     `json:"user_id"`
    Username    string   `json:"username"`
    Role        string   `json:"role"`
    Permissions []string `json:"permissions"`
    jwt.RegisteredClaims
}

// RBAC Permission Check
func CheckPermission(permissions []string, required string) bool {
    for _, p := range permissions {
        if p == "*" || p == required {
            return true
        }
    }
    return false
}
```

Slide 4: Source Code Structure

Go Project Structure

Directory Layout

```
internal/
├── api/
│   ├── v1/                    # 24 REST handler files (9,814 LOC)
│   │   ├── auth.go            # Authentication endpoints
│   │   ├── site.go            # Website management
│   │   ├── database.go        # Database CRUD
│   │   ├── docker.go          # Docker management (46+ endpoints)
│   │   ├── file.go            # File browser/editor
│   │   ├── docker_compose.go  # Compose projects
│   │   ├── ssl.go             # SSL certificates
│   │   ├── backup.go          # Backup management
```

```
│   │   ├── security.go         # Firewall, SSH, Fail2ban
│   │   ├── cron.go             # Cron jobs
│   │   ├── log.go              # Log viewer
│   │   ├── monitor.go          # System metrics
│   │   ├── user.go             # User management
│   │   ├── role.go             # RBAC roles
│   │   ├── settings.go         # Panel settings
│   │   ├── software.go         # Software installation
│   │   ├── ftp.go              # FTP accounts
│   │   ├── ftp_storage.go      # FTP backup storage
│   │   ├── cloud_storage.go    # S3/GDrive/OneDrive
│   │   ├── redis.go            # Redis management
│   │   ├── task.go             # Background tasks
│   │   ├── notification.go     # Notifications
│   │   ├── plugin.go           # Plugin system
│   │   ├── waf.go              # WAF management
│   │   ├── system.go           # System info
│   │   └── docker_cleanup.go   # Docker auto-cleanup
│   │
│   └── ws/                     # 2 WebSocket handlers
│       ├── terminal.go         # PTY terminal (xterm.js ↔ creack/pty)
│       └── monitor.go          # Real-time metrics stream
│
├── app/                        # Application bootstrap
│   ├── app.go                  # Main entry, graceful shutdown
│   ├── router.go               # Gin route registration
│   ├── auth_middleware.go      # JWT + RBAC middleware
│   └── middleware.go           # CORS, rate limit, compression
│
├── service/                    # Business logic (23,437 LOC)
│   ├── auth_service.go         # Login, register, 2FA
│   ├── jwt.go                  # Token generation/validation
│   ├── totp_service.go         # TOTP 2FA (RFC 6238)
│   ├── user_service.go         # User CRUD
│   ├── role_service.go         # RBAC role management
│   ├── site_service.go         # Website CRUD + config
│   ├── site_statistics_service.go  # Traffic statistics
│   ├── database_service.go     # MySQL/MariaDB management
│   ├── file_service.go         # File operations
│   ├── docker_service.go       # Docker API wrapper
│   ├── docker_compose_service.go   # Compose project management
│   ├── docker_cleanup_service.go    # Auto-cleanup old images/containers
│   ├── backup_service.go       # Backup orchestration
│   ├── backup_incremental.go    # Incremental backup logic
│   ├── backup_encryption.go    # AES-256 encryption
│   ├── backup_upload_service.go    # Multi-destination upload
│   ├── ssl_service.go          # Let's Encrypt integration
│   ├── ssl_renewer.go          # Auto-renewal scheduler
│   ├── cron_service.go         # Cron job management
```

```
│   ├── terminal_service.go     # PTY session management
│   ├── monitor_service.go      # System metrics collection
│   ├── metrics_collector.go    # CPU/RAM/Disk/Network
│   ├── security_service.go     # SSH, Fail2ban, hardening
│   ├── firewall_service.go     # UFW rules management
│   ├── software_service.go     # Software install/uninstall
│   ├── settings_service.go     # Panel configuration
│   ├── redis_service.go        # Redis management
│   ├── ftp_service.go         # PureFTPd management
│   ├── ftp_storage_service.go  # FTP storage backends
│   ├── cloud_storage_service.go # S3/GDrive/OneDrive
│   ├── notification_service.go # Email/Telegram/Webhook
│   ├── email_service.go        # SMTP integration
│   ├── telegram_service.go     # Telegram bot notifications
│   ├── geo_service.go          # GeoIP lookup
│   ├── log_service.go         # Log aggregation
│   ├── waf_service.go         # WAF rule management
│   ├── waf_log_service.go     # WAF access logs
│   ├── haproxy_manager.go      # HAProxy configuration
│   ├── coraza_spoa.go         # Coraza WAF SPOA integration
│   ├── permission_cache.go     # RBAC permission caching
│   ├── user_resource_service.go    # User quotas
│   ├── site_defaults.go       # Default site configuration
│   │
│   ├── installer/             # 14 software installers
│   │   ├── installer.go       # Base installer interface
│   │   ├── nginx.go          # Nginx installation
│   │   ├── apache.go         # Apache installation
│   │   ├── ols.go            # OpenLiteSpeed
│   │   ├── php.go            # PHP (multiple versions)
│   │   ├── mysql.go          # MySQL installation
│   │   ├── postgresql.go     # PostgreSQL
│   │   ├── mongodb.go        # MongoDB
│   │   ├── redis.go         # Redis
│   │   ├── memcached.go     # Memcached
│   │   ├── phpmyadmin.go     # phpMyAdmin
│   │   ├── pgadmin.go       # pgAdmin
│   │   ├── pureftpd.go       # PureFTPd
│   │   ├── supervisor.go     # Supervisor
│   │   ├── docker.go         # Docker Engine
│   │   ├── nodejs.go         # Node.js
│   │   ├── python.go         # Python
│   │   ├── golang.go         # Go
│   │   ├── java.go           # Java/Tomcat
│   │   ├── elasticsearch.go   # Elasticsearch
│   │   ├── s3_storage.go     # MinIO S3
│   │   ├── google_drive.go   # Google Drive OAuth
│   │   ├── onedrive.go       # OneDrive OAuth
│   │   └── ftp_storage.go    # FTP storage
```

```
│   │
│   └── vhost/                # 4 web server vhost generators
│       ├── vhost.go          # Base vhost interface
│       ├── nginx.go          # Nginx site config
│       ├── apache.go         # Apache vhost
│       └── ols.go            # OpenLiteSpeed vhost
│
├── database/                 # SQLite + models
│   ├── sqlite.go             # Database connection pool
│   ├── models/               # Data models
│   │   └── role.go           # RBAC models
│   └── migrations/           # 14 migration files
│       ├── migrations.go     # Migration runner
│       ├── 002_monitor_metrics.go
│       ├── 003_firewall_settings.go
│       ├── 004_phpmyadmin_default_port.go
│       ├── 005_quota.go
│       ├── 006_site_description.go
│       ├── 007_permissions.go
│       ├── 008_panel_settings.go
│       ├── 009_ftp_storage.go
│       ├── 010_cloud_storages.go
│       ├── 011_user_resources.go
│       ├── 012_notifications.go
│       ├── 013_notification_templates.go
│       ├── 014_site_type.go
│       ├── 015_docker_cleanup.go
│       ├── 016_waf.go
│       ├── 017_waf_enhance.go
│       ├── 018_backup_uploads.go
│       ├── 019_cron_storage.go
│       └── 020_incremental_backups.go
│
├── config/                   # Configuration
│   └── config.go             # JSON config reader/writer
│
├── pkg/                      # Shared utilities
│   └── ...                   # Crypto, logger, errors
│
├── i18n/                     # Backend i18n
│   └── ...                   # EN/VI translations
│
├── task/                     # Background task queue
│   └── queue.go              # Async task with progress
│
├── websocket/                # WebSocket hub
│   └── ...                   # Connection management
│
├── system/                   # System utilities
```

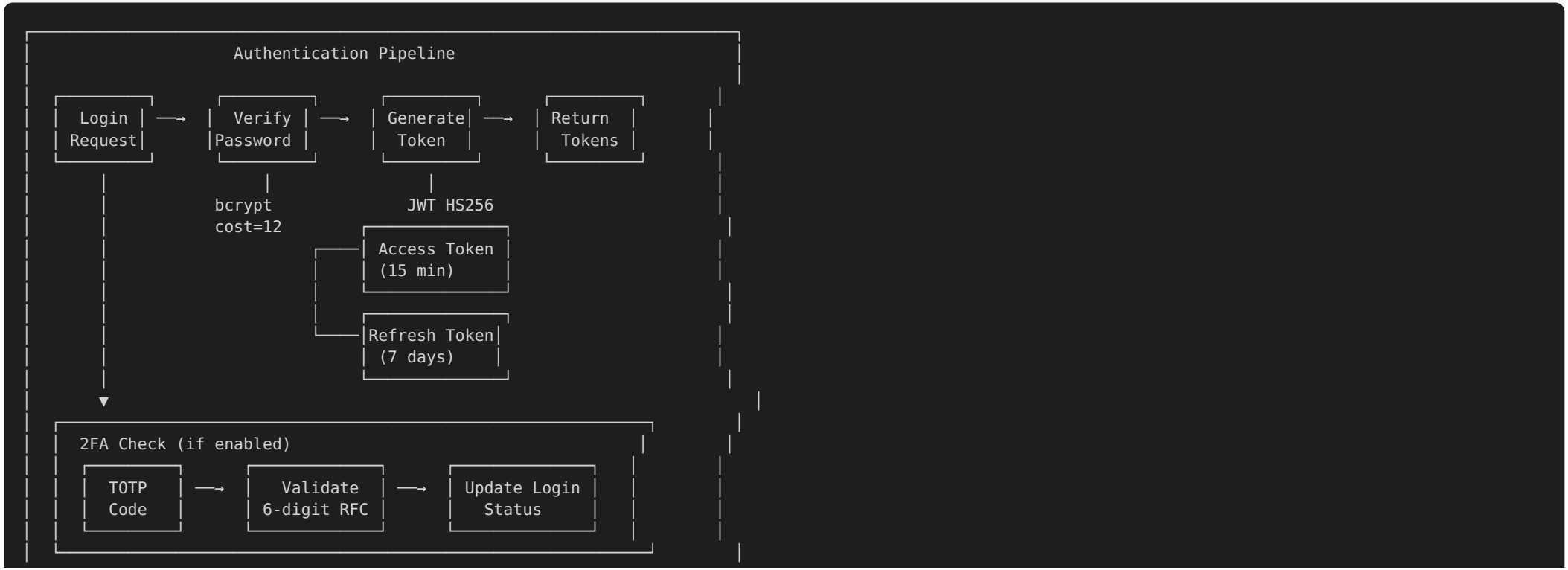
```
│   ├── shell.go            # Shell execution
│   └── os_detect.go        # OS detection
│
└── plugin/                 # Plugin system
    ├── plugin.go           # Plugin interface
    └── manager.go          # Plugin loader

cmd/
└── panel/
    └── main.go             # Entry point
```

Slide 5: Authentication & Authorization

JWT + TOTP 2FA + RBAC

Authentication Flow



```
└──────────────────────────────────────────────────────────────┘
```

JWT Token Structure

```
// Access Token Claims (15 minutes TTL)
type AccessClaims struct {
    UserID      uint     `json:"user_id"`
    Username    string   `json:"username"`
    Email       string   `json:"email"`
    Role        string   `json:"role"`
    Permissions []string `json:"permissions"`  // Cached permissions
    TokenType   string   `json:"type"`         // "access"
    RegisteredClaims
}

// Refresh Token Claims (7 days TTL)
type RefreshClaims struct {
    UserID    uint   `json:"user_id"`
    TokenType string `json:"type"`  // "refresh"
    jti       string `json:"jti"`  // Unique token ID for revocation
    RegisteredClaims
}
```

RBAC Permission Model

```
┌──────────────────────────────────────────────────────────────┐
│  Permission Namespace System                                 │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Admin Role: ["*"]                                   │    │
│  │  ├── Full access to all resources                    │    │
│  │  └── Can manage users and roles                      │    │
│  └──────────────────────────────────────────────────────┘    │
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Operator Role:                                      │    │
│  │  ├── sites.*           (all site operations)         │    │
│  │  ├── databases.*       (all database operations)     │    │
│  │  ├── files.*           (all file operations)         │    │
│  │  ├── docker.*         (all docker operations)        │    │
│  │  ├── cron.*            (all cron operations)         │    │
│  │  ├── backup.*         (all backup operations)        │    │
│  │  └── terminal.access  (terminal access)              │    │
│  └──────────────────────────────────────────────────────┘    │
```

```
│                                                              │
│  ┌──────────────────────────────────────────────────────┐    │
│  │  Viewer Role:                                        │    │
│  │  ├── sites.read         (read-only site view)        │    │
│  │  ├── databases.read     (read-only database view)    │    │
│  │  ├── files.read         (read-only file view)        │    │
│  │  ├── logs.read          (read-only log view)         │    │
│  │  └── docker.read        (read-only docker view)      │    │
│  └──────────────────────────────────────────────────────┘    │
└──────────────────────────────────────────────────────────────┘
```

TOTP 2FA Implementation

```
// TOTP Configuration (RFC 6238)
const (
    TOTPIssuer      = "ITDLUPanel"
    TOTPDigits      = 6
    TOTPPeriod      = 30  // seconds
    TOTPSecretSize  = 20  // bytes
)

// Enable 2FA - Generate QR Code Secret
func (s *AuthService) EnableTOTP(userID uint) (*TOTPResponse, error) {
    // Generate random secret
    secret, err := totp.Generate(totp.GenerateOpts{
        Issuer:      TOTPIssuer,
        AccountName: username,
        SecretSize:  TOTPSecretSize,
        Period:      TOTPPeriod,
        Digits:      otp.DigitsSix,
        Algorithm:   otp.AlgorithmSHA1,
    })

    // Generate provisioning URI for QR code
    provisioningURI := secret.URL()

    return &TOTPResponse{
        Secret:     secret.Secret(),
        ProvisioningURI: provisioningURI,
        QRCode:     generateQRCodeBase64(provisioningURI),
    }
}

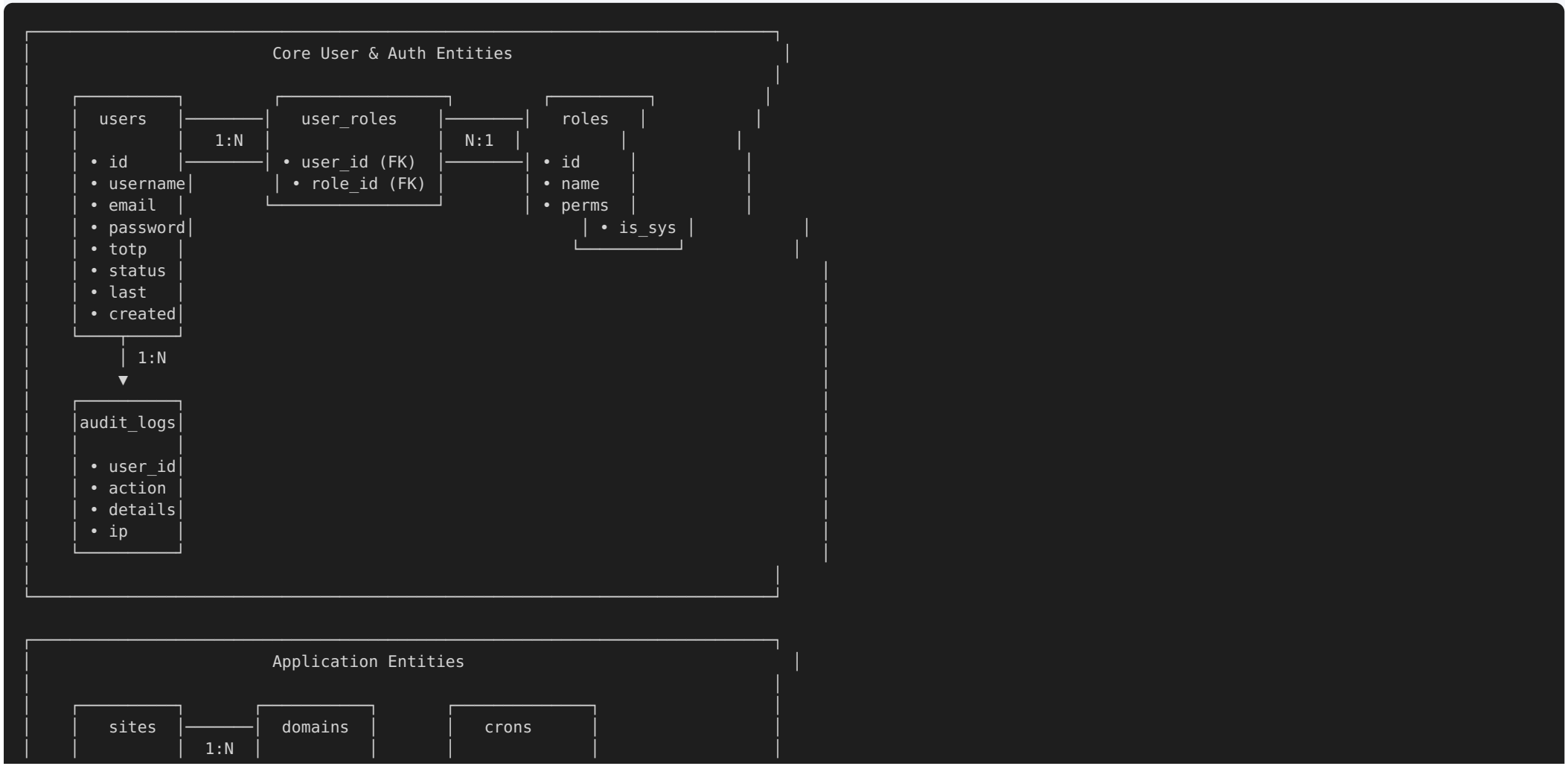
// Verify TOTP Code
func (s *AuthService) VerifyTOTP(secret, code string) bool {
    return totp.Validate(code, secret)
```

```
}
```

Slide 6: Database Schema Chi Tiết

SQLite Database Architecture

Entity Relationship Diagram




• domain
• root
• php
• server
• ssl
• domain
• type
• name
• schedule
• command
• last_run
• status
1:N
databases
• name
• type
• user
• pass
ftps
• username
• path
• status
tasks
• name
• type
• status
• progress
• message
Storage & Backup Entities
backups
• name
• type
• target
• file
• size
• storage
N:1
ftp_storages
• host
• port
• username
• path
N:1
cloud_storages
• type
• config
(encrypted)
Docker & Notification Entities
docker containers
• name
• image
• status
notification_ch
• name
• type
• config
• enabled
1:N
notification_rules
• event
• threshold
• enabled
1:N



Key Table Definitions

Users Table

```
CREATE TABLE users (
    id              INTEGER PRIMARY KEY AUTOINCREMENT,
    username        TEXT UNIQUE NOT NULL,
    email           TEXT UNIQUE,
    password_hash   TEXT NOT NULL,          -- bcrypt cost=12
    totp_secret     TEXT,                    -- base32 encoded
    totp_enabled    INTEGER DEFAULT 0,
    status          INTEGER DEFAULT 1,       -- 1=active, 0=disabled
    last_login      DATETIME,
    created_at      DATETIME DEFAULT CURRENT_TIMESTAMP,
    updated_at      DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- Indexes
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_email ON users(email);
```

Roles Table

```
CREATE TABLE roles (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    name        TEXT UNIQUE NOT NULL,
    description TEXT,
    permissions TEXT NOT NULL,              -- JSON array ["sites.*", "docker.*"]
    is_system   INTEGER DEFAULT 0,          -- 1=system role (cannot delete)
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- System roles
INSERT INTO roles (name, permissions, is_system) VALUES
```

```
    ('admin', '["*"]', 1),
    ('operator', '["sites.*","databases.*","files.*","docker.*","cron.*","backup.*","terminal.access"]', 1),
    ('viewer', '["sites.read","databases.read","files.read","logs.read","docker.read"]', 1);
```

Audit Logs Table

```
CREATE TABLE audit_logs (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id     INTEGER REFERENCES users(id),
    username    TEXT,
    action      TEXT NOT NULL,              -- CREATE, UPDATE, DELETE, LOGIN, etc.
    resource    TEXT,                       -- sites, databases, users, etc.
    details     TEXT,                       -- JSON with additional details
    ip_address  TEXT,
    created_at  DATETIME DEFAULT CURRENT_TIMESTAMP
);

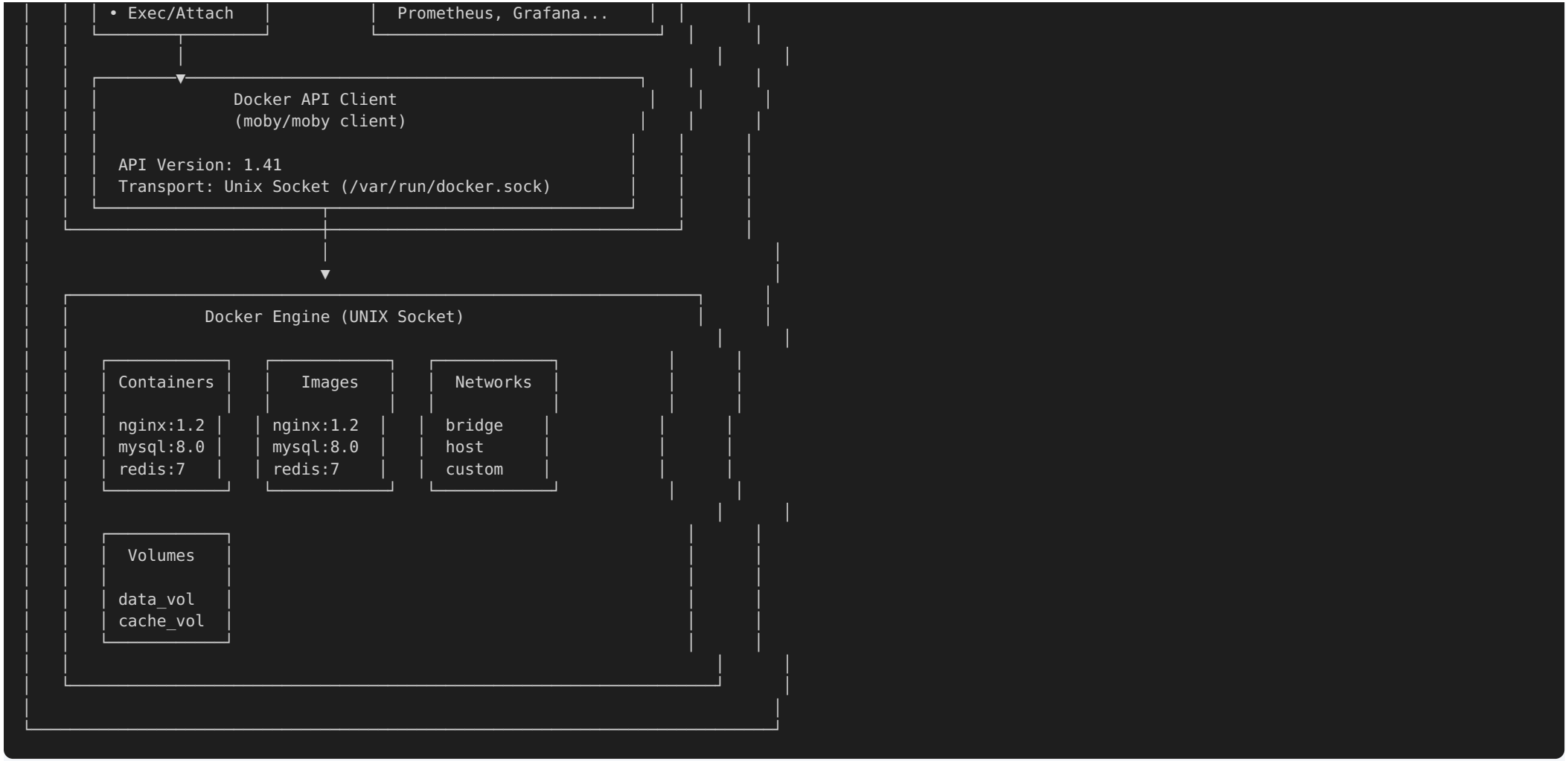
-- Index for efficient querying
CREATE INDEX idx_audit_user ON audit_logs(user_id);
CREATE INDEX idx_audit_created ON audit_logs(created_at);
CREATE INDEX idx_audit_action ON audit_logs(action);
```

Slide 7: Docker Integration Chi Tiết

Docker Management System

Architecture Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                    Docker Management Architecture                    │
│                                                                      │
│   ┌──────────────────────────────────────────────────────┐      │
│   │                   ITDLUPanel Backend                          │      │
│   │                                                       │      │
│   │   ┌─────────────────┐          ┌──────────────────────────┐  │      │
│   │   │ DockerService   │          │  OneClick App Templates  │  │      │
│   │   │                 │          │  (38+ pre-configured)    │  │      │
│   │   │ • List/Inspect  │          │                          │  │      │
│   │   │ • Start/Stop    │          │  WordPress, Nextcloud,   │  │      │
│   │   │ • Logs/Stats    │          │  GitLab, Matomo, Jira,   │  │      │
```



Docker API Endpoints

Container Management (20+ endpoints)

```
// Container CRUD
GET    /api/v1/docker/containers             // List all containers
POST   /api/v1/docker/containers             // Create container
GET    /api/v1/docker/containers/:id         // Inspect container
DELETE /api/v1/docker/containers/:id         // Remove container

// Container Lifecycle
```

```
POST   /api/v1/docker/containers/:id/start     // Start container
POST   /api/v1/docker/containers/:id/stop      // Stop container
POST   /api/v1/docker/containers/:id/restart    // Restart container
POST   /api/v1/docker/containers/:id/pause      // Pause container
POST   /api/v1/docker/containers/:id/unpause    // Unpause container

// Container Operations
GET    /api/v1/docker/containers/:id/logs    // Get container logs
GET    /api/v1/docker/containers/:id/stats   // Get resource stats
POST   /api/v1/docker/containers/:id/rename  // Rename container
POST   /api/v1/docker/containers/:id/exec    // Execute command
POST   /api/v1/docker/containers/:id/commit  // Create image from container
```

Image Management (6 endpoints)

```
GET    /api/v1/docker/images                // List all images
POST   /api/v1/docker/images/pull           // Pull image from registry
DELETE /api/v1/docker/images/:id           // Remove image
GET    /api/v1/docker/images/:id           // Inspect image
GET    /api/v1/docker/images/search        // Search Docker Hub
POST   /api/v1/docker/images/:id/tag       // Tag image
```

Network Management (7 endpoints)

```
GET    /api/v1/docker/networks              // List networks
POST   /api/v1/docker/networks             // Create network
DELETE /api/v1/docker/networks/:id          // Remove network
GET    /api/v1/docker/networks/:id          // Inspect network
POST   /api/v1/docker/networks/:id/connect    // Connect container
POST   /api/v1/docker/networks/:id/disconnect // Disconnect container
```

Volume Management (5 endpoints)

```
GET    /api/v1/docker/volumes              // List volumes
POST   /api/v1/docker/volumes             // Create volume
DELETE /api/v1/docker/volumes/:name       // Remove volume
GET    /api/v1/docker/volumes/:name       // Inspect volume
POST   /api/v1/docker/volumes/prune       // Prune unused volumes
```

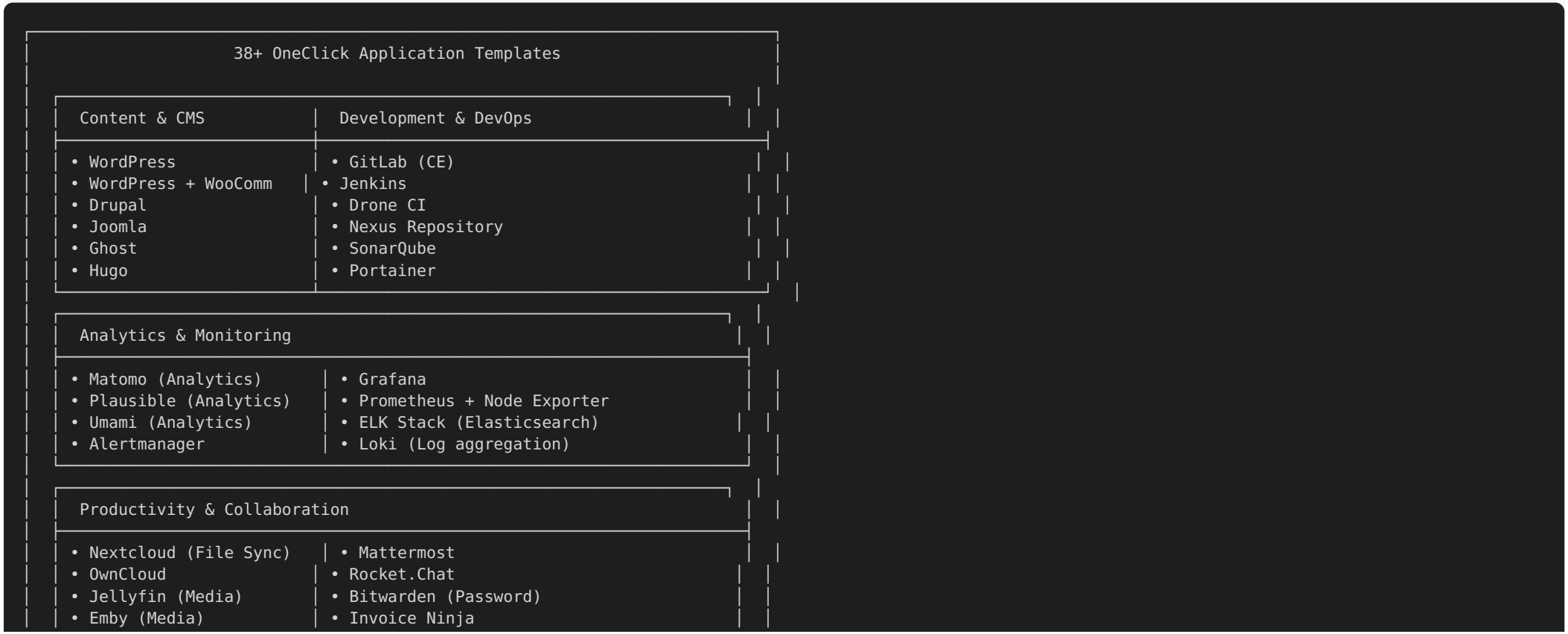
Docker Compose (5 endpoints)

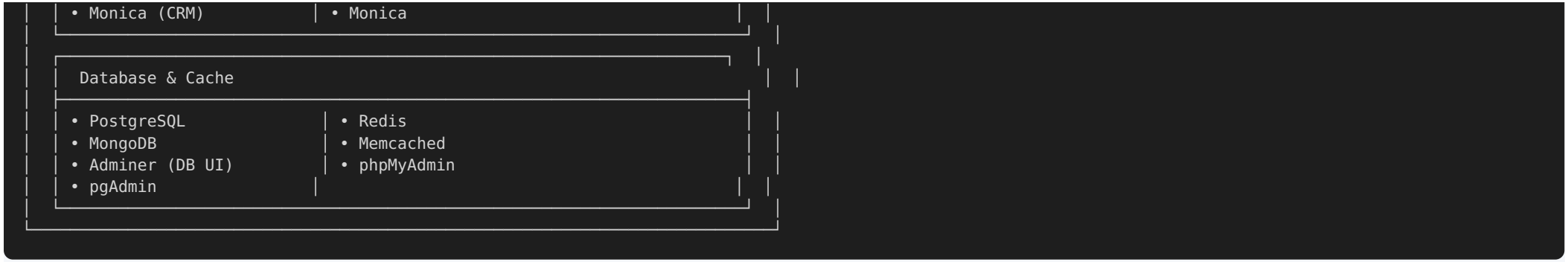
```
GET    /api/v1/docker/compose              // List compose projects
POST   /api/v1/docker/compose              // Create compose project
POST   /api/v1/docker/compose/:name/up     // Start project
POST   /api/v1/docker/compose/:name/down   // Stop project
DELETE /api/v1/docker/compose/:name        // Remove project
```

System (5 endpoints)

```
GET    /api/v1/docker/info                 // Docker engine info
GET    /api/v1/docker/daemon/config        // Get daemon.json
POST   /api/v1/docker/daemon/config        // Save daemon.json
POST   /api/v1/docker/restart              // Restart Docker
POST   /api/v1/docker/stop                 // Stop Docker
```

OneClick App Templates

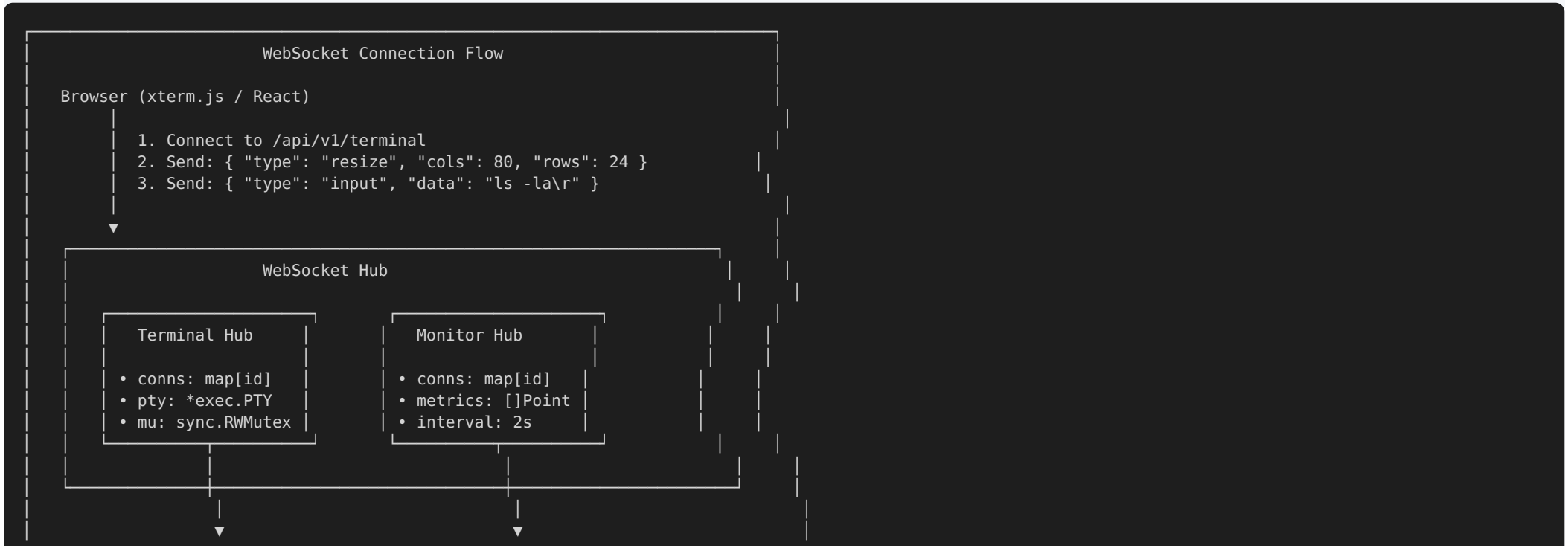




Slide 8: WebSocket Real-Time Communication

WebSocket Architecture

Dual WebSocket Endpoints



```
│   ┌──────────────────────────────────┐  ┌──────────────────────────────────┐      │
│   │      Terminal PTY Service        │  │     Monitor Service              │      │
│   │                                  │  │                                  │      │
│   │  • creack/pty for pseudo-TTY     │  │  • gopsutil for metrics          │      │
│   │  • Shell: /bin/bash              │  │  • CPU, RAM, Disk, Network       │      │
│   │  • Window resize handling        │  │  • 2-second intervals            │      │
│   │  • Buffer scrollback             │  │  • Buffered history              │      │
│   └──────────────────────────────────┘  └──────────────────────────────────┘      │
│                                                                                  │
└──────────────────────────────────────────────────────────────────────────────────┘
```

Terminal WebSocket Protocol

```
// Client → Server Messages
type TerminalInput struct {
    Type string `json:"type"`  // "input"
    Data string `json:"data"`  // Keystrokes
}

type TerminalResize struct {
    Type  string `json:"type"`  // "resize"
    Cols  int    `json:"cols"`
    Rows  int    `json:"rows"`
}

// Server → Client Messages
type TerminalOutput struct {
    Type string `json:"type"`  // "output"
    Data string `json:"data"`  // Terminal output (base64 or raw)
}

// Backend Implementation
func (h *TerminalHub) HandleWebSocket(c *gin.Context) {
    // Upgrade HTTP to WebSocket
    conn, err := upgrader.Upgrade(c.Writer, c.Request, nil)
    defer conn.Close()

    // Create PTY
    f, tty, err := pty.StartWithSize(cmd, nil)
    defer f.Close()

    // Bidirectional copy
    go func() {
        io.Copy(f, conn)  // Client → PTY
    }()
    io.Copy(conn, f)     // PTY → Client
```

```
}
```

Monitor WebSocket Protocol

```
// Server → Client (every 2 seconds)
type MetricsUpdate struct {
    Type     string    `json:"type"`      // "metrics"
    Timestamp time.Time `json:"timestamp"`
    CPU       CPUMetrics  `json:"cpu"`
    Memory    MemMetrics  `json:"memory"`
    Disk      DiskMetrics `json:"disk"`
    Network   NetMetrics  `json:"network"`
}

type CPUMetrics struct {
    Percent   float64   `json:"percent"`   // 0-100
    User      float64   `json:"user"`
    System    float64   `json:"system"`
    Idle      float64   `json:"idle"`
    CoreCount int       `json:"core_count"`
}

type MemMetrics struct {
    Total       uint64  `json:"total"`      // bytes
    Used        uint64  `json:"used"`
    Available   uint64  `json:"available"`
    UsedPercent float64 `json:"used_percent"`
}

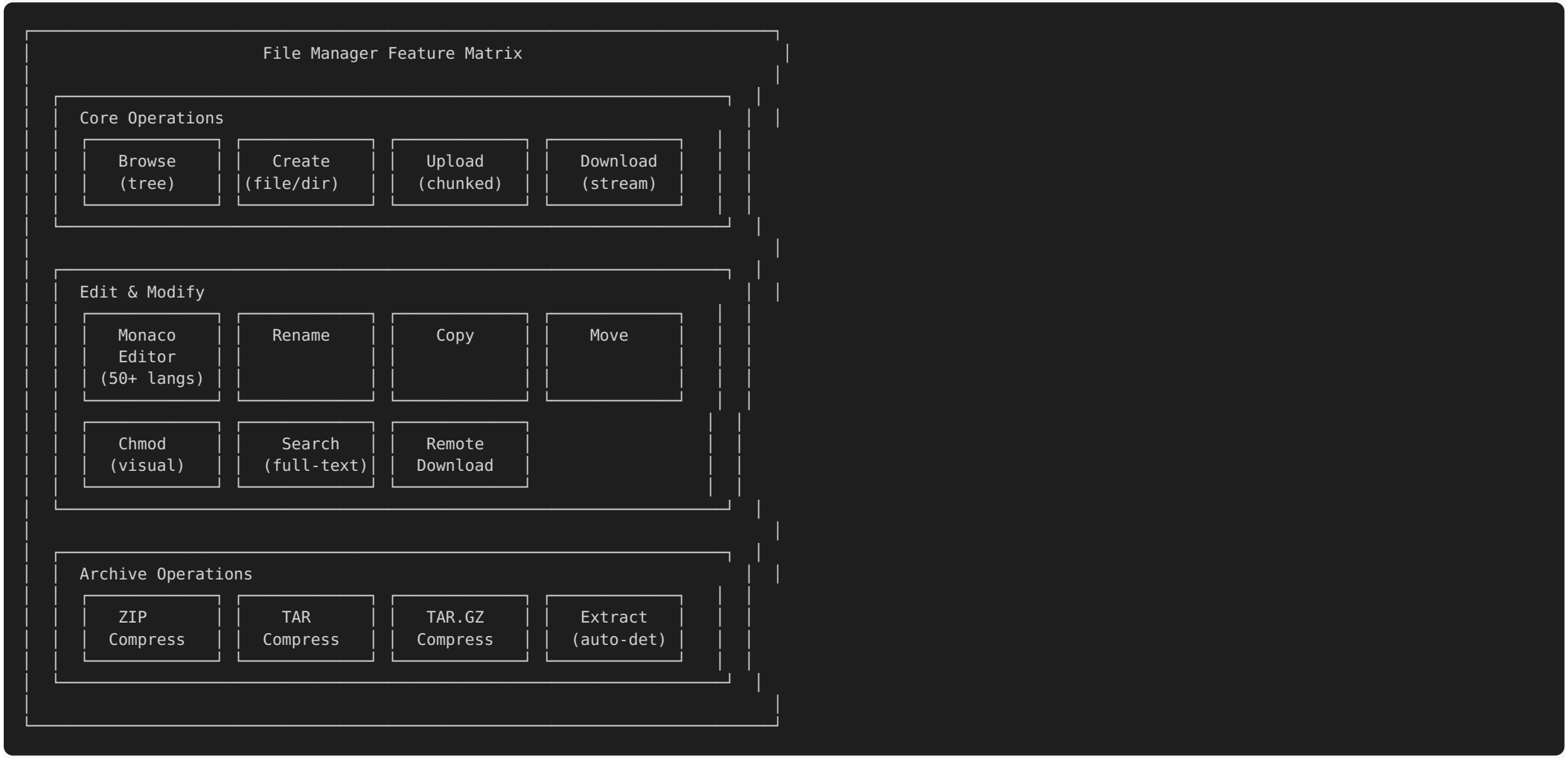
type DiskMetrics struct {
    Total       uint64  `json:"total"`
    Used        uint64  `json:"used"`
    Free        uint64  `json:"free"`
    UsedPercent float64 `json:"used_percent"`
}

type NetMetrics struct {
    BytesSent   uint64 `json:"bytes_sent"`
    BytesRecv   uint64 `json:"bytes_recv"`
    PacketsSent uint64 `json:"packets_sent"`
    PacketsRecv uint64 `json:"packets_recv"`
}
```

Slide 9: File Management System

File Manager Architecture

Capabilities Overview



Monaco Editor Integration

```
// Monaco Editor Supported Languages (50+)
var supportedLanguages = []string{
    // Web
    "javascript", "typescript", "html", "css", "scss", "less",
    "json", "xml", "yaml", "markdown",

    // Backend
    "go", "python", "java", "c", "cpp", "csharp", "ruby", "php", "rust",

    // Config
    "ini", "properties", "dotenv", "nginx", "apache",

    // Shell
    "shell", "powershell", "dockerfile",

    // Database
    "sql", "plsql", "mysql",

    // Data
    "csv", "toml",
}
```

File API Endpoints

```
// Navigation & Info
GET  /api/v1/files                  // List directory contents
GET  /api/v1/files/info             // Get file/directory info
GET  /api/v1/files/tree            // Get directory tree

// Read & Write
GET  /api/v1/files/read            // Read file content
POST /api/v1/files/save            // Save file content

// Create & Delete
POST /api/v1/files/create           // Create file or directory
DELETE /api/v1/files                // Delete files/directories

// Modify
POST /api/v1/files/rename          // Rename file/directory
POST /api/v1/files/copy            // Copy file/directory
POST /api/v1/files/move            // Move file/directory
POST /api/v1/files/permissions     // Change permissions (chmod)

// Transfer
POST /api/v1/files/upload          // Upload file (multipart)
```

```
GET  /api/v1/files/download        // Download file
POST /api/v1/files/remote-download // Download from URL

// Archive
POST /api/v1/files/compress        // Compress to ZIP/TAR/TARGZ
POST /api/v1/files/decompress      // Extract archive

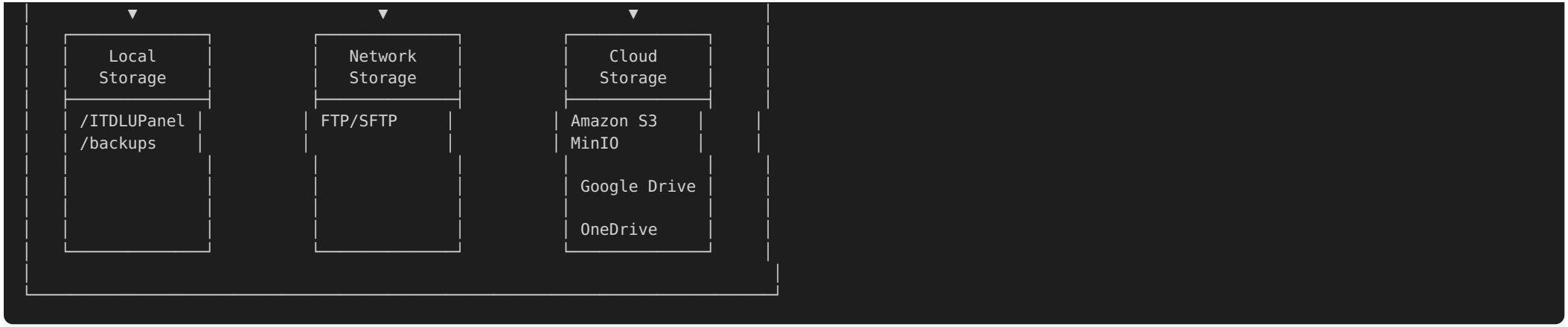
// Search
POST /api/v1/files/search          // Search files
```

Slide 10: Backup System Architecture

Multi-Destination Backup System

Backup Flow





Backup Types

Full Backup

```
# Database (MySQL)
mysqldump --single-transaction --routines --triggers \
    --events --all-databases | gzip > backup_full.sql.gz

# Files
tar -czf backup_files.tar.gz /var/www/sites/
```

Incremental Backup

```
// Incremental backup using rsync-style diff
type IncrementalBackup struct {
    BaseBackupID  uint      `db:"base_backup_id"`
    DiffPath      string    `db:"diff_path"`
    ChangesCount  int       `db:"changes_count"`
    ChangesSize   int64     `db:"changes_size"`
    CreatedAt     time.Time `db:"created_at"`
}

// Only backup changed files since last backup
func (s *BackupService) CreateIncremental(ctx context.Context, siteID uint) error {
    lastBackup := s.getLastBackup(siteID)
    lastBackupTime := lastBackup.CreatedAt

    // Find all modified files
    changes, err := s.findChanges(siteID, lastBackupTime)
```

```
    // Create diff archive
    for _, change := range changes {
        s.addToDiff(change)
    }

    return nil
}
```

Encryption

```
// AES-256-GCM Encryption
type EncryptedBackup struct {
    Ciphertext   []byte
    Nonce        []byte    // 12 bytes
    Tag          []byte    // 16 bytes
}

// Encrypt backup
func EncryptBackup(data []byte, password string) (*EncryptedBackup, error) {
    key := deriveKey(password)  // PBKDF2 with 100,000 iterations

    block, err := aes.NewCipher(key)
    gcm, err := cipher.NewGCM(block)

    nonce := make([]byte, gcm.NonceSize())
    io.ReadFull(rand.Reader, nonce)

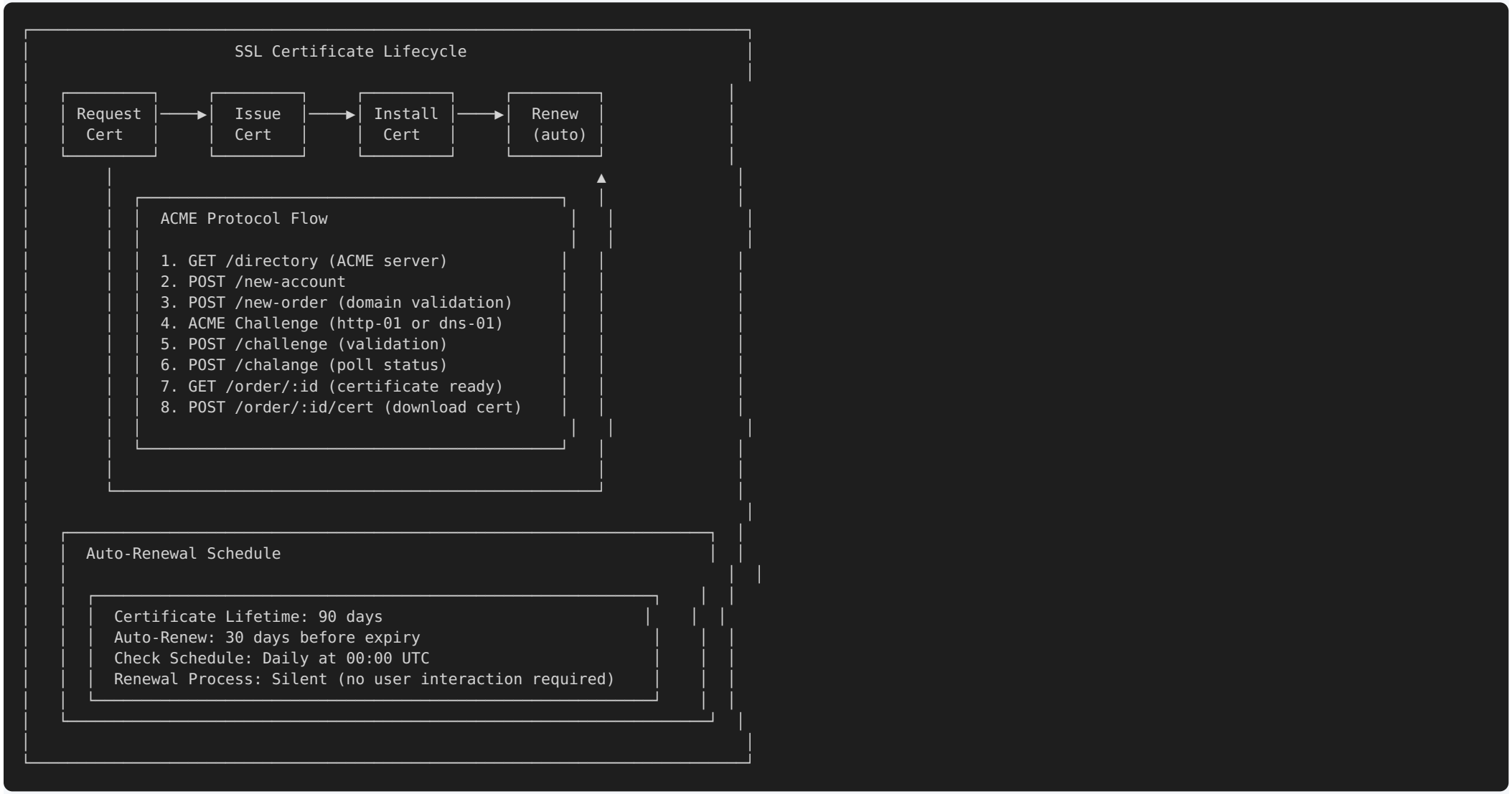
    sealed := gcm.Seal(nonce, nonce, data, nil)

    return &EncryptedBackup{
        Ciphertext: sealed[len(nonce):],
        Nonce:      nonce,
        Tag:        sealed[len(nonce):],
    }, nil
}
```

Slide 11: SSL Certificate Management

Let's Encrypt Integration

Certificate Lifecycle



Certificate Storage

```
// Certificate Storage Structure
type SSLCertificate struct {
    Domain         string    `json:"domain"`
    CertPath       string    `json:"cert_path"`       // /etc/ssl/certs/domain.crt
```

```
    KeyPath        string    `json:"key_path"`        // /etc/ssl/private/domain.key
    ChainPath      string    `json:"chain_path"`      // /etc/ssl/certs/domain.chain.crt
    Provider       string    `json:"provider"`        // "letsencrypt"
    ValidFrom      time.Time `json:"valid_from"`
    ValidUntil     time.Time `json:"valid_until"`
    AutoRenew      bool      `json:"auto_renew"`
    LastRenewed    time.Time `json:"last_renewed"`
}

// Nginx Configuration
const nginxSSLConfig = `
server {
    listen 443 ssl http2;
    server_name {{.Domain}};

    ssl_certificate     {{.CertPath}};
    ssl_certificate_key {{.KeyPath}};
    ssl_trusted_certificate {{.ChainPath}};

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256;
    ssl_prefer_server_ciphers on;
    ssl_session_cache shared:SSL:10m;

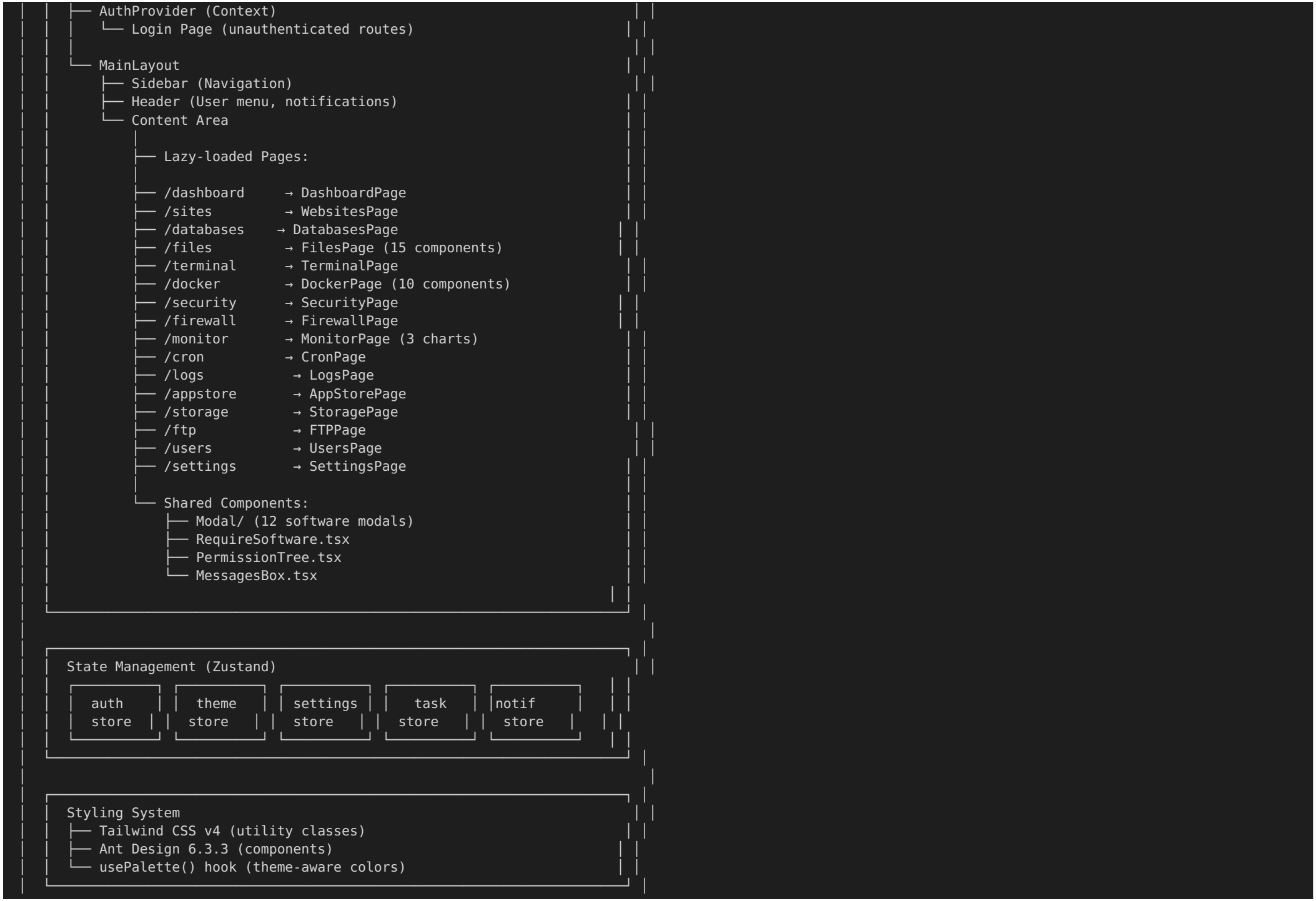
    # Security headers
    add_header Strict-Transport-Security "max-age=31536000" always;
    add_header X-Frame-Options DENY;
    add_header X-Content-Type-Options nosniff;
}
`
```

Slide 12: Frontend Architecture

React 19 SPA Architecture

Component Hierarchy

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Frontend Architecture                         │
│                                                                      │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  App.tsx (Root)                                              │    │
```


├── AuthProvider (Context)
└── Login Page (unauthenticated routes)
└── MainLayout
├── Sidebar (Navigation)
├── Header (User menu, notifications)
└── Content Area
├── Lazy-loaded Pages:
├── /dashboard → DashboardPage
├── /sites → WebsitesPage
├── /databases → DatabasesPage
├── /files → FilesPage (15 components)
├── /terminal → TerminalPage
├── /docker → DockerPage (10 components)
├── /security → SecurityPage
├── /firewall → FirewallPage
├── /monitor → MonitorPage (3 charts)
├── /cron → CronPage
├── /logs → LogsPage
├── /appstore → AppStorePage
├── /storage → StoragePage
├── /ftp → FTPPage
├── /users → UsersPage
├── /settings → SettingsPage
└── Shared Components:
├── Modal/ (12 software modals)
├── RequireSoftware.tsx
├── PermissionTree.tsx
└── MessagesBox.tsx
State Management (Zustand)
auth store
theme store
settings store
task store
notif store
Styling System
├── Tailwind CSS v4 (utility classes)
├── Ant Design 6.3.3 (components)
└── usePalette() hook (theme-aware colors)

```
│                                                                                │
└────────────────────────────────────────────────────────────────────────────────┘
```

Key React Patterns

State Management

```
// Zustand Store Example
interface AuthState {
  user: User | null;
  accessToken: string | null;
  refreshToken: string | null;
  isAuthenticated: boolean;

  // Actions
  login: (credentials: LoginCredentials) => Promise<void>;
  logout: () => void;
  refreshTokens: () => Promise<void>;
  updateProfile: (data: Partial<User>) => void;
}

export const useAuthStore = create<AuthState>((set, get) => ({
  user: null,
  accessToken: null,
  refreshToken: null,
  isAuthenticated: false,

  login: async (credentials) => {
    const response = await api.post('/auth/login', credentials);
    const { user, access_token, refresh_token } = response.data;

    set({
      user,
      accessToken: access_token,
      refreshToken: refresh_token,
      isAuthenticated: true
    });

    // Store in localStorage for persistence
    localStorage.setItem('refresh_token', refresh_token);
  },

  logout: () => {
    api.post('/auth/logout');
    set({ user: null, accessToken: null, refreshToken: null, isAuthenticated: false });
    localStorage.removeItem('refresh_token');
```

```
  },
}));
```

Theme System

```
// usePalette hook
export function usePalette() {
  const isDark = document.body.getAttribute('data-theme') === 'dark';

  return {
    isDark,
    p: {
      bg: isDark ? '#0D1117' : '#F5F7FA',
      card: isDark ? '#161B22' : '#FFFFFF',
      cardBorder: isDark ? 'rgba(255,255,255,0.1)' : '#E8EDF2',
      text: isDark ? '#E6EDF3' : '#1A1A2E',
      textMuted: isDark ? '#8B949E' : '#64748B',
      headerBg: isDark ? '#21262D' : '#FAFBFC',
      accent: '#1565C0',
      green: '#52C41A',
      red: '#FF4D4F',
      amber: '#FAAD14',
      purple: '#722ED1',
      cyan: '#13C2C2',
    }
  };
}
```

Slide 13: Performance Optimization

Performance & Optimization

Backend Optimizations

Database Connection Pooling

```
// SQLite connection pool settings
type DBConfig struct {
    MaxOpenConns    int = 25        // Maximum open connections
    MaxIdleConns    int = 5         // Maximum idle connections
```

```
    ConnMaxLifetime time.Duration = time.Hour
}

// Prepared statements for frequently used queries
type PreparedStatements struct {
    GetUserByUsername *sql.Stmt
    GetSitesByUser    *sql.Stmt
    GetContainers     *sql.Stmt
    InsertAuditLog    *sql.Stmt
}
```

Concurrent Request Handling

```
// Gin context pool for memory optimization
engine.Use(gin.Logger())
engine.Use(gin.Recovery())

// Custom context pool
engine.Use(func(c *gin.Context) {
    c.Next()
})

// HTTP server settings
s := &http.Server{
    Addr:         ":8888",
    Handler:      engine,
    ReadTimeout:  30 * time.Second,
    WriteTimeout: 30 * time.Second,
    IdleTimeout:  120 * time.Second,
    MaxHeaderBytes: 1 << 20, // 1MB
}
```

Caching Strategy

```
// Permission cache with TTL
type PermissionCache struct {
    mu    sync.RWMutex
    data  map[uint][]string
    timer map[uint]*time.Timer
    ttl   time.Duration
}

func (c *PermissionCache) Get(userID uint) ([]string, bool) {
    c.mu.RLock()
    defer c.mu.RUnlock()
```

```
    perms, ok := c.data[userID]
    return perms, ok
}

func (c *PermissionCache) Set(userID uint, perms []string) {
    c.mu.Lock()
    defer c.mu.Unlock()

    c.data[userID] = perms

    // Reset timer
    if c.timer[userID] != nil {
        c.timer[userID].Stop()
    }
    c.timer[userID] = time.AfterFunc(c.ttl, func() {
        c.Delete(userID)
    })
}
```

Frontend Optimizations

Code Splitting

```
// React.lazy for all pages
const Dashboard = lazy(() => import('./pages/Dashboard'));
const Websites = lazy(() => import('./pages/Websites'));
const Docker = lazy(() => import('./pages/Docker'));
const Files = lazy(() => import('./pages/Files'));

// Route-level splitting
<Suspense fallback={<Spin />}>
  <Routes>
    <Route path="/dashboard" element={<Dashboard />} />
    <Route path="/docker/*" element={<Docker />} />
  </Routes>
</Suspense>
```

Memoization

```
// Heavy component memoization
const HeavyTable = memo(({ data, columns }: TableProps) => {
  return (
    <Table
      dataSource={data}
```

```
      columns={useMemo(() => columns, [columns])}
      pagination={false}
    />
  );
});

// Expensive computation memoization
const processedData = useMemo(() => {
  return data.map(item => transformItem(item));
}, [data]);
```

WebSocket Optimization

```
// Debounced metrics updates
const MetricsUpdater = () => {
  const [metrics, setMetrics] = useState<Metrics>(null);

  useEffect(() => {
    let buffer: Metrics[] = [];
    let rafId: number;

    const processBuffer = () => {
      // Take latest from buffer, ignore others
      if (buffer.length > 0) {
        setMetrics(buffer[buffer.length - 1]);
        buffer = [];
      }
      rafId = requestAnimationFrame(processBuffer);
    };

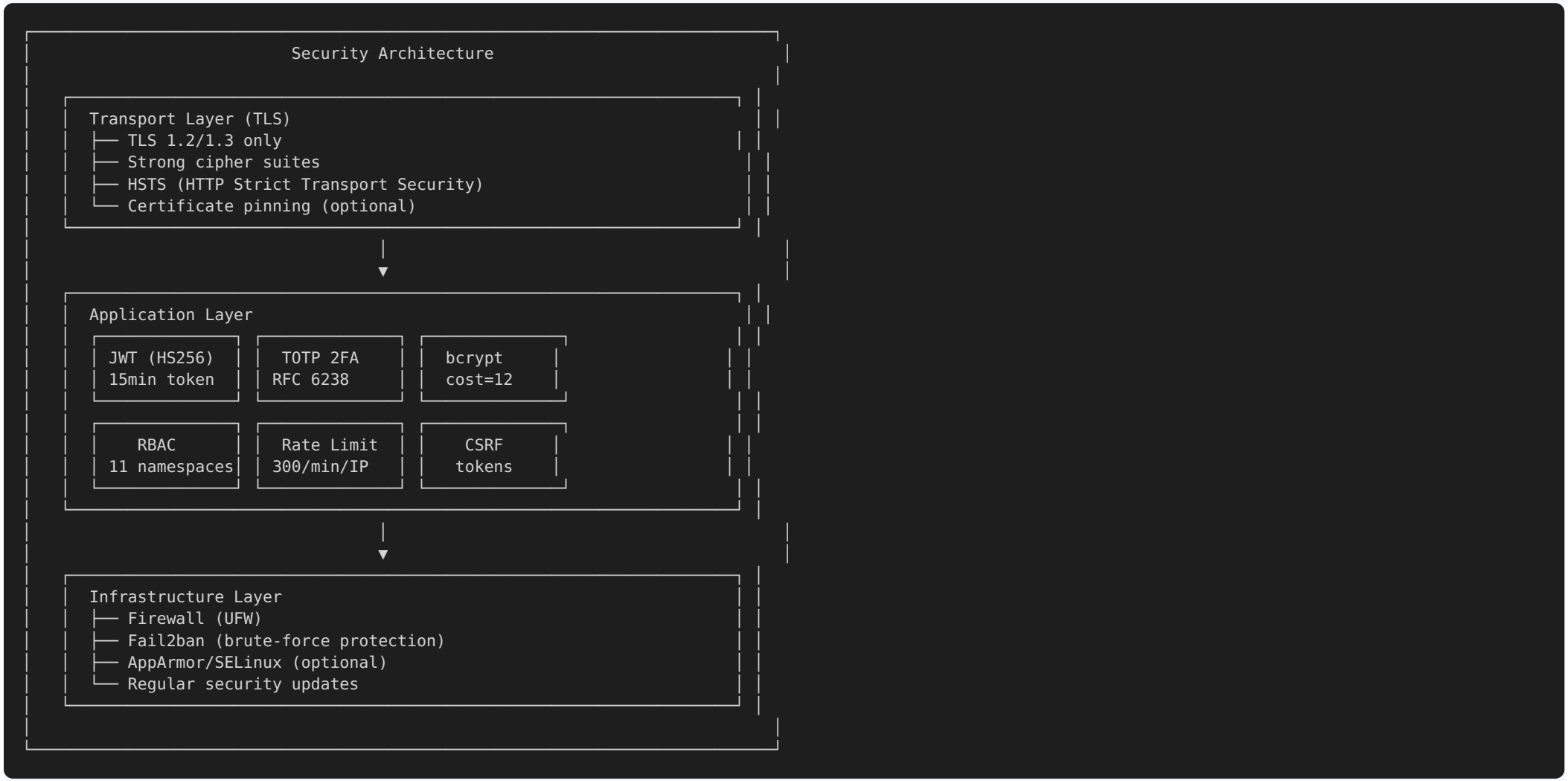
    ws.onmessage = (event) => {
      buffer.push(JSON.parse(event.data));
    };

    rafId = requestAnimationFrame(processBuffer);
    return () => cancelAnimationFrame(rafId);
  }, []);
};
```

Slide 14: Security Implementation

Security Hardening

Security Layers



Security Headers

```
// Middleware: SecurityHeaders
func SecurityHeadersMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        // Prevent clickjacking
        c.Header("X-Frame-Options", "DENY")
```

```
        // Prevent MIME sniffing
        c.Header("X-Content-Type-Options", "nosniff")

        // XSS filter (legacy but still useful)
        c.Header("X-XSS-Protection", "1; mode=block")

        // Content Security Policy
        c.Header("Content-Security-Policy",
            "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline';")

        // Referrer Policy
        c.Header("Referrer-Policy", "strict-origin-when-cross-origin")

        c.Next()
    }
}
```

Password Security

```
// bcrypt with cost=12
const bcryptCost = 12

func HashPassword(password string) (string, error) {
    bytes, err := bcrypt.GenerateFromPassword([]byte(password), bcryptCost)
    return string(bytes), err
}

func CheckPassword(password, hash string) bool {
    err := bcrypt.CompareHashAndPassword([]byte(hash), []byte(password))
    return err == nil
}

// Password requirements
const passwordMinLength = 8
const passwordRequireUppercase = true
const passwordRequireLowercase = true
const passwordRequireNumber = true
const passwordRequireSpecial = true
```

Rate Limiting

```
// Token bucket rate limiter
type RateLimiter struct {
```

```
    tokens     float64
    maxTokens  float64
    refillRate float64  // tokens per second
    lastRefill time.Time
    mu         sync.Mutex
}

func (rl *RateLimiter) Allow() bool {
    rl.mu.Lock()
    defer rl.mu.Unlock()

    // Refill tokens
    now := time.Now()
    elapsed := now.Sub(rl.lastRefill).Seconds()
    rl.tokens = min(rl.maxTokens, rl.tokens+elapsed*rl.refillRate)
    rl.lastRefill = now

    // Check if request allowed
    if rl.tokens >= 1 {
        rl.tokens--
        return true
    }
    return false
}

// Configuration: 300 requests per minute = 5 requests per second
rateLimiter := &RateLimiter{
    maxTokens:  300,
    refillRate: 5,
}
```

Slide 15: CI/CD & Deployment

Build & Deployment Pipeline

Build Process

```
┌──────────────────────────────────────────────────────────────────────┐
│                        Build Pipeline                                │
│                                                                      │
│   ┌──────────────────────────────────────────────────────────────┐   │
│   │  Phase 1: Frontend Build (Node.js 22)                        │   │
```



Deployment Options

1. Direct Binary

```
# Download and install
wget https://github.com/itdlu/ITDLUPanel/releases/latest/itdlupanel
chmod +x itdlupanel
sudo mv itdlupanel /usr/local/bin/

# Create systemd service
sudo tee /etc/systemd/system/itdlupanel.service <<EOF
[Unit]
Description=ITDLUPanel Server
After=network.target

[Service]
Type=simple
User=root
WorkingDirectory=/ITDLUPanel
ExecStart=/usr/local/bin/itdlupanel -config /ITDLUPanel/config/panel.json
```

```
Restart=always
RestartSec=10

[Install]
WantedBy=multi-user.target
EOF

# Enable and start
sudo systemctl daemon-reload
sudo systemctl enable itdlupanel
sudo systemctl start itdlupanel
```

2. Docker Deployment

```
# Dockerfile
FROM alpine:3.21

RUN apk add --no-cache ca-certificates tzdata

COPY itdlupanel /usr/local/bin/
RUN chmod +x /usr/local/bin/itdlupanel

EXPOSE 8888

USER 1000

ENTRYPOINT ["itdlupanel"]
CMD ["-config", "/data/panel.json"]

VOLUME ["/data"]
```

```
# Run with Docker
docker run -d \
  --name itdlupanel \
  -p 8888:8888 \
  -v /ITDLUPanel/data:/data \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --restart unless-stopped \
  itdlu/itdlupanel:latest
```

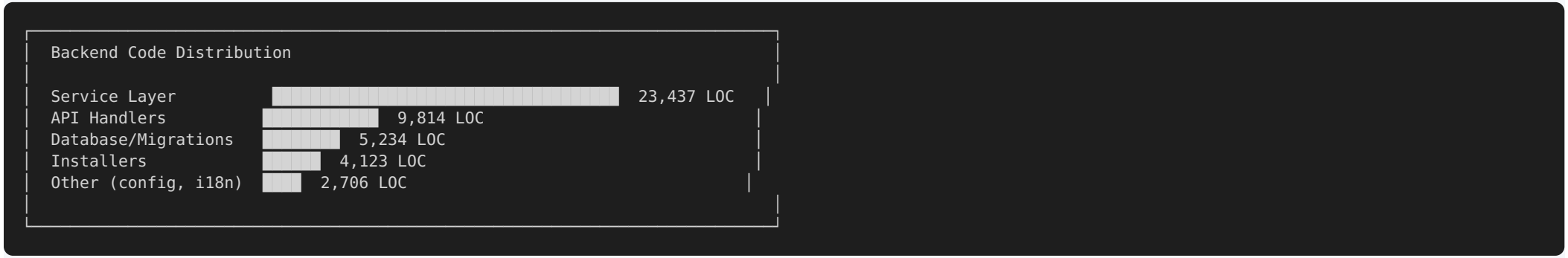
Slide 16: Code Statistics

Project Metrics

Backend (Go)

Metric	Count
Total Go Files	112
Total Lines of Code	43,314
API Handlers	24 files
Services	33+
Installers	14
Vhost Generators	4
Database Migrations	14

LOC Breakdown by Layer

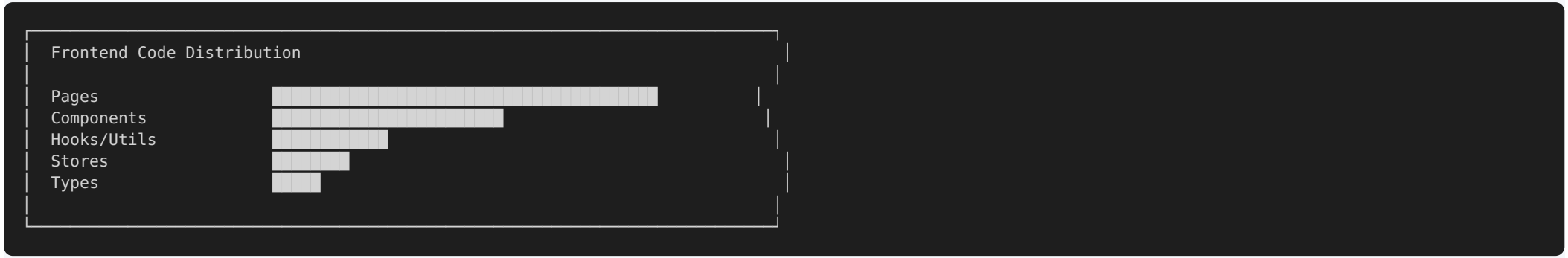


Frontend (React/TypeScript)

Metric	Count
Total TSX/TS Files	93

Pages	17 (lazy-loaded)
Shared Components	15
Zustand Stores	5
Custom Hooks	7+

Frontend Code Distribution



API Coverage

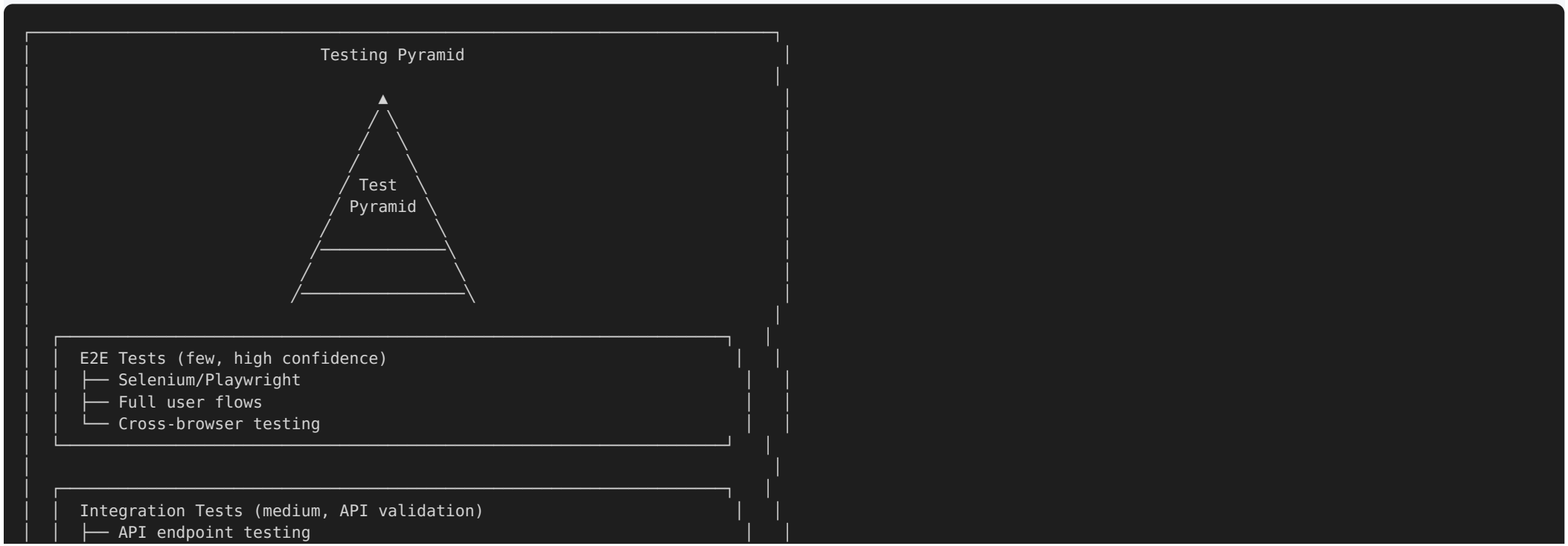
Module	REST Endpoints	WebSocket
Auth	9	-
Dashboard	2	-
Sites	14	-
Databases	8	-
Docker	46+	-
Files	17	-
Security	12	-
Cron	8	-
Logs	7	-

SSL	9	-
Backup	6	-
Monitor	5	1
Terminal	-	1
Total	**~160**	**2**

Slide 17: Testing Strategy

Quality Assurance

Testing Pyramid



```
│  │  ├── Database operations                                          │  │
│  │  └── Service interactions                                         │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                        │
│  ┌──────────────────────────────────────────────────────────────────┐  │
│  │  Unit Tests (many, fast feedback)                                 │  │
│  │  ├── Service layer functions                                      │  │
│  │  ├── Utility functions                                            │  │
│  │  └── Model validation                                             │  │
│  └──────────────────────────────────────────────────────────────────┘  │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Backend Testing

```go
// Example: Service unit test
func TestSiteService_Create(t *testing.T) {
    db := setupTestDB(t)
    service := NewSiteService(db)

    // Test successful creation
    site, err := service.Create(&CreateSiteRequest{
        Domain:   "example.com",
        RootPath: "/var/www/example",
        PHPVer:   "8.2",
        WebServ:  "nginx",
    })

    require.NoError(t, err)
    require.NotNil(t, site)
    require.Equal(t, "example.com", site.Domain)

    // Test duplicate domain
    _, err = service.Create(&CreateSiteRequest{
        Domain: "example.com",
    })
    require.Error(t, err)
    require.Equal(t, ErrDomainExists, err)
}

// Example: Integration test
func TestDockerAPI_ListContainers(t *testing.T) {
    if testing.Short() {
        t.Skip("Skipping integration test")
    }
```

```
    // Requires running Docker daemon
    service := NewDockerService()
    containers, err := service.ListContainers()

    require.NoError(t, err)
    require.NotNil(t, containers)
}
```

Frontend Testing

```
// Example: Component test with React Testing Library
describe('SiteCard', () => {
  it('renders site information correctly', () => {
    const site = {
      id: 1,
      domain: 'example.com',
      php_version: '8.2',
      status: 'running'
    };

    render(<SiteCard site={site} onEdit={jest.fn()} />);

    expect(screen.getByText('example.com')).toBeInTheDocument();
    expect(screen.getByText('PHP 8.2')).toBeInTheDocument();
  });

  it('calls onEdit when edit button is clicked', () => {
    const onEdit = jest.fn();
    render(<SiteCard site={site} onEdit={onEdit} />);

    fireEvent.click(screen.getByRole('button', { name: /edit/i }));

    expect(onEdit).toHaveBeenCalledWith(1);
  });
});

// Example: Hook test
describe('useMetrics', () => {
  it('fetches and updates metrics', async () => {
    const { result } = renderHook(() => useMetrics());

    // Initial state
    expect(result.current.loading).toBe(true);

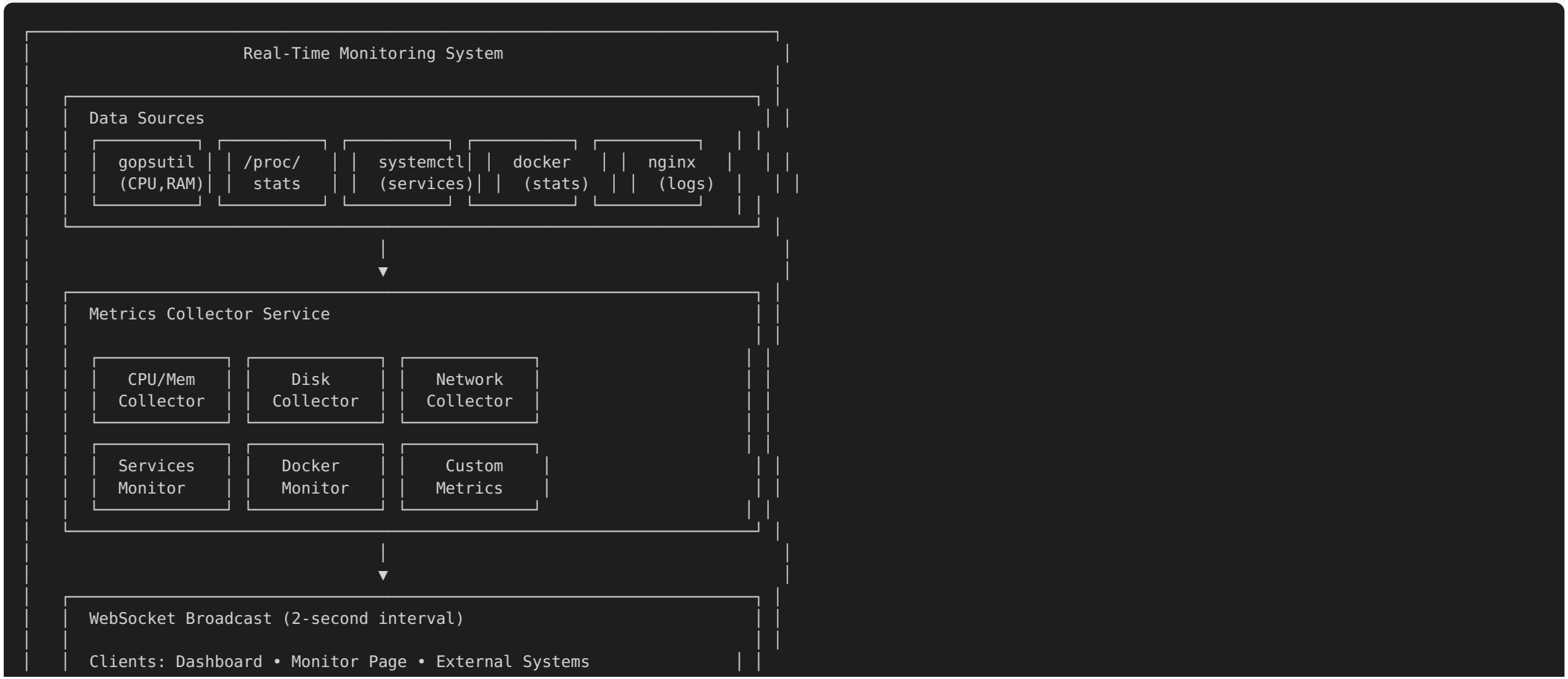
    // Wait for data
    await waitFor(() => {
```

```
      expect(result.current.loading).toBe(false);
      expect(result.current.metrics).toBeDefined();
    });
  });
});
```

Slide 18: Monitoring & Observability

System Monitoring

Metrics Collection



```
│    └──────────────────────────────────────────────────────────────────┘ │
│                                                                        │
└────────────────────────────────────────────────────────────────────────┘
```

Metrics API

```
// System metrics endpoint
type SystemMetrics struct {
    Timestamp time.Time  `json:"timestamp"`

    CPU       CPUMetrics  `json:"cpu"`
    Memory    MemMetrics  `json:"memory"`
    Disk      DiskMetrics `json:"disk"`
    Network   NetMetrics  `json:"network"`
    Services  []ServiceStatus `json:"services"`
    Uptime    time.Duration   `json:"uptime"`
}

type CPUMetrics struct {
    Percent    float64   `json:"percent"`
    User       float64   `json:"user"`
    System     float64   `json:"system"`
    Idle       float64   `json:"idle"`
    CoreCount  int       `json:"core_count"`
    PerCore    []float64 `json:"per_core"`
}

type MemMetrics struct {
    Total       uint64  `json:"total"`        // bytes
    Used        uint64  `json:"used"`
    Available   uint64  `json:"available"`
    Cached      uint64  `json:"cached"`
    UsedPercent float64 `json:"used_percent"`
}

type DiskMetrics struct {
    Partitions []Partition `json:"partitions"`
    IOStats    IOStats      `json:"io_stats"`
}

type NetMetrics struct {
    BytesSent   uint64    `json:"bytes_sent"`
    BytesRecv   uint64    `json:"bytes_recv"`
    PacketsSent uint64    `json:"packets_sent"`
    PacketsRecv uint64    `json:"packets_recv"`
    ErrIn      uint64    `json:"err_in"`
```

```
    ErrOut     uint64    `json:"err_out"`
    PerInterface []IFaceStats `json:"per_interface"`
}
```

Slide 19: Error Handling & Logging

Logging & Error Management

Structured Logging

```
// zerolog configuration
import "github.com/rs/zerolog"

func initLogger() zerolog.Logger {
    // Console writer with color
    output := zerolog.ConsoleWriter{Out: os.Stdout, TimeFormat: time.RFC3339}

    logger := zerolog.New(output).
        Level(zerolog.InfoLevel).
        With().
        Timestamp().
        Caller().
        Str("service", "itdlupanel").
        Logger()

    return logger
}

// Usage with structured logging
log.Info().
    Str("user", username).
    Str("action", "login").
    Str("ip", clientIP).
    Msg("User logged in")

log.Error().
    Err(err).
    Str("site_id", "123").
    Str("operation", "backup").
    Msg("Backup failed")
```

Error Types

```
// Custom error types
type AppError struct {
    Code    string `json:"code"`
    Message string `json:"message"`
    Details string `json:"details,omitempty"`
    Status  int    `json:"-"`
}

var (
    ErrNotFound = &AppError{
        Code:    "NOT_FOUND",
        Message: "Resource not found",
        Status:  404,
    }

    ErrUnauthorized = &AppError{
        Code:    "UNAUTHORIZED",
        Message: "Authentication required",
        Status:  401,
    }

    ErrForbidden = &AppError{
        Code:    "FORBIDDEN",
        Message: "Permission denied",
        Status:  403,
    }

    ErrValidation = &AppError{
        Code:    "VALIDATION_ERROR",
        Message: "Invalid request data",
        Status:  400,
    }

    ErrInternal = &AppError{
        Code:    "INTERNAL_ERROR",
        Message: "Internal server error",
        Status:  500,
    }
)

// Error handling middleware
func ErrorHandler() gin.HandlerFunc {
    return func(c *gin.Context) {
        c.Next()
```

```
        if len(c.Errors) > 0 {
            err := c.Errors.Last()
            appErr, ok := err.Err.(*AppError)
            if !ok {
                appErr = ErrInternal
            }

            c.JSON(appErr.Status, gin.H{
                "error": appErr.Code,
                "message": appErr.Message,
                "details": appErr.Details,
            })
        }
    }
}
```

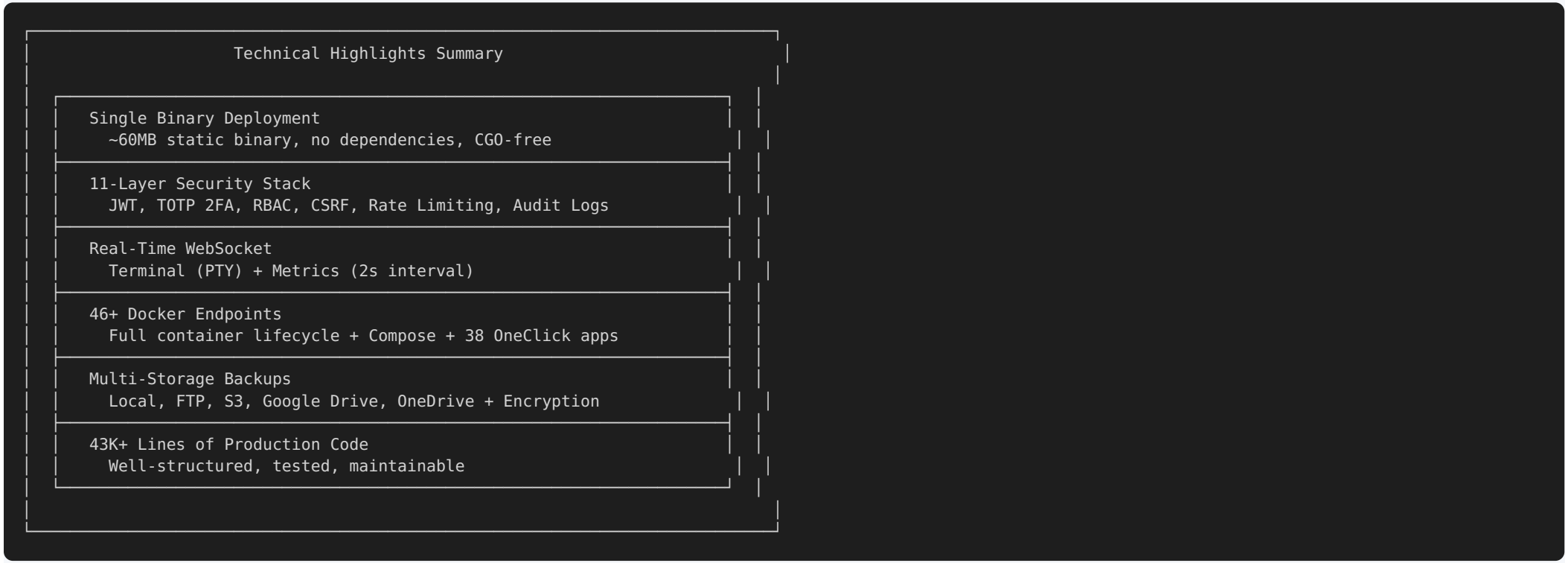
Audit Log Format

```
{
  "id": 12345,
  "user_id": 1,
  "username": "admin",
  "action": "UPDATE",
  "resource": "sites",
  "details": {
    "site_id": 5,
    "domain": "example.com",
    "changes": {
      "php_version": {"old": "8.1", "new": "8.2"},
      "ssl_enabled": {"old": false, "new": true}
    }
  },
  "ip_address": "192.168.1.100",
  "user_agent": "Mozilla/5.0...",
  "created_at": "2026-06-30T14:25:03Z"
}
```

Slide 20: Kết Luận & Roadmap

Tổng Kết

Những Điểm Nổi Bật Kỹ Thuật



Phase 1: Completed

- [x] Core panel functionality (112 Go files, 43K LOC)
- [x] Docker management (46+ endpoints, 38 OneClick apps)
- [x] WebSocket terminal (xterm.js ↔ PTY)
- [x] SSL certificates (Let's Encrypt auto-renewal)
- [x] Multi-language (EN/VI)
- [x] RBAC with 3 roles
- [x] TOTP 2FA
- [x] Incremental backups
- [x] File manager with Monaco Editor

Phase 2: In Progress

- [ ] Kubernetes cluster management
- [ ] Advanced monitoring (Prometheus/Grafana)
- [ ] Plugin marketplace
- [ ] API documentation (OpenAPI/Swagger)
- [ ] WAF (Coraza + HAProxy)

Phase 3: Planned

- [ ] Multi-server management
- [ ] Mobile app (React Native)
- [ ] AI-powered suggestions
- [ ] GitOps integration

Liên Hệ

- **Website**: https://github.com/itdlu/ITDLUPanel
- **Email**: support@itdlu.edu.vn
- **Docs**: https://docs.itdlu.edu.vn

© 2026 Đại học Đà Lạt (Dalat University)

Generated by ITDLUPanel Documentation System - Technical Deep Dive v2